



Testes e ajustes

Desenvolvimento Móvel

Aula 32/36

Flutter

Widget

Provider

Json

Model

ios

Teste

Conceito

Testes são fundamentais para qualidade do app. Testes de widget com `flutter_test`: use `pumpWidget` para renderizar, `find` para localizar widgets por texto, tipo ou Key, e `expect` para verificar comportamentos. Testes de integração com `integration_test`: automatizam fluxos completos (login -> navegar -> preencher formulário -> verificar resultado). Testes unitários: focam em modelos (`fromJson/toJson`), controllers (`Cubit/Provider`), e repositories (mock de dependências). Cobertura de teste: `flutter test --coverage` gera `lcov.info`, visualizável com `genhtml`. Ajustes finais de UI: responsividade (testar em diferentes tamanhos), tema (consistência de cores, dark mode), animações (performance com DevTools timeline) e acessibilidade (labels, contraste, font scaling).

Piramide de Testes

Integracao

Testes de Widget

Testes Unitarios (mockito/mocktail)

`flutter test --coverage`



Atividade Prática

Escreva: (1) testes unitarios para 2 repositorios usando mock de datasources (mockito ou mocktail), (2) testes de widget para 2 telas principais (ex: login e home), (3) 1 teste de integracao simulando fluxo completo. Execute `flutter test --coverage`, gere o relatorio HTML e analise a cobertura. Corrija qualquer teste falhando.



Pesquisa

Pesquise o pacote mockito ou mocktail para Dart: como criar mocks tipados, definir comportamentos com `when().thenReturn()`, verificar interacoes com `verify()`, e usar Mocktail (sem geracao de codigo) como alternativa mais simples.



Requisitos

Flutter SDK. Adicionar `flutter_test` (nativo), `mockito/mocktail` e `integration_test` ao `pubspec.yaml`.